

E-Commerce Customer Churn Analysis Report

Adithya C. V.

Dataset

[E-Commerce Customer Churn Analysis](#)

Objective

To clean, transform, and analyze the customer churn dataset using MySQL. The objective is to handle missing and inconsistent data, remove outliers, perform data transformations, and analyze customer behavior, churn patterns, complaints, orders, payments, and returns to derive useful insights.

Database Used

“ecomm” is the database used for the customer churn and customer returns analysis.

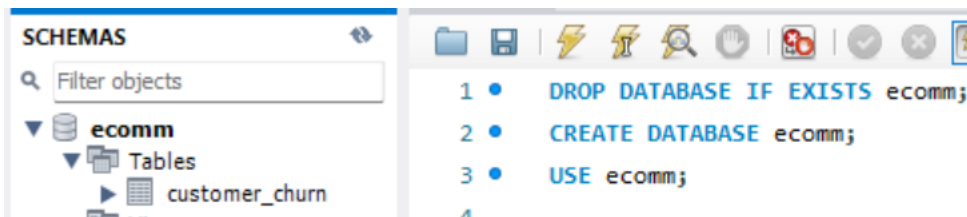


Table Used

customer_churn – Existing table containing the customer churn dataset.

```
CREATE TABLE customer_churn(  
  CustomerID INT PRIMARY KEY,  
  Churn BIT,  
  Tenure INT,  
  PreferredLoginDevice VARCHAR(20),  
  CityTier INT,  
  WarehouseToHome INT,  
  PreferredPaymentMode VARCHAR(20),  
  Gender ENUM('Male', 'Female'),  
  HourSpendOnApp INT,  
  NumberOfDeviceRegistered INT,  
  PreferredOrderCat VARCHAR(20),  
  SatisfactionScore INT,  
  MaritalStatus VARCHAR(10),  
  NumberOfAddress INT,  
  Complain BIT,  
  OrderAmountHikeFromlastYear INT,  
  CouponUsed INT,  
  OrderCount INT,  
  DaySinceLastOrder INT,  
  CashbackAmount INT  
);
```

The customer_churn table contains customer details such as CustomerID, Churn, Tenure, PreferredLoginDevice, CityTier, WarehouseToHome, PreferredPaymentMode, Gender, HourSpendOnApp, NumberOfDeviceRegistered, PreferredOrderCat, SatisfactionScore, MaritalStatus, NumberOfAddress, Complain, OrderAmountHikeFromlastYear, CouponUsed, OrderCount, DaySinceLastOrder, and CashbackAmount.

```

28 • INSERT INTO customer_churn(CustomerID,Churn,Tenure,PreferredLoginDevice,CityTier,WarehouseToHome,PreferredPaymentMode,Gender,HourSp
29 (50001,1,4,'Mobile Phone',3,6,'Debit Card','Female',3,3,'Laptop & Accessory',2,'Single',9,1,11,1,1,5,160),
30 (50002,1,NULL,'Phone',1,8,'UPI','Male',3,4,'Mobile',3,'Single',7,1,15,0,1,0,121),
31 (50003,1,NULL,'Phone',1,30,'Debit Card','Male',2,4,'Mobile',3,'Single',6,1,14,0,1,3,120),
32 (50004,1,0,'Phone',3,15,'Debit Card','Male',2,4,'Laptop & Accessory',5,'Single',8,0,23,0,1,3,134),
33 (50005,1,0,'Phone',1,12,'CC','Male',NULL,3,'Mobile',5,'Single',3,0,11,1,1,3,130),
34 (50006,1,0,'Computer',1,22,'Debit Card','Female',3,5,'Mobile Phone',5,'Single',2,1,22,4,6,7,139),
35 (50007,1,NULL,'Phone',3,11,'Cash on Delivery','Male',2,3,'Laptop & Accessory',2,'Divorced',4,0,14,0,1,0,121),
36 (50008,1,NULL,'Phone',1,6,'CC','Male',3,3,'Mobile',2,'Divorced',3,1,16,2,2,0,123),
37 (50009,1,13,'Phone',3,9,'E wallet','Male',NULL,4,'Mobile',3,'Divorced',2,1,14,0,1,2,127),

```

Data Cleaning

1. Handling Missing Values and Outliers:

Mean Imputation:

- **Impute Mean for WarehouseToHome:** The null values in the WarehouseToHome column were replaced with the mean value of 16.

```
-- Mean for WarehouseToHome --
UPDATE customer_churn
SET WarehouseToHome = (
    SELECT ROUND(AVG(WarehouseToHome))
    FROM (SELECT WarehouseToHome FROM customer_churn) AS temp
)
WHERE WarehouseToHome IS NULL;
```

WarehouseToHome	WarehouseToHome
NULL	<u>16</u>
17	17
NULL	<u>16</u>
11	11
6	6

- Impute Mean for HourSpendOnApp: The null values in the HourSpendOnApp column were replaced with the mean value of 3.

```
-- Mean for HourSpendOnApp --
UPDATE customer_churn
SET HourSpendOnApp = (
    SELECT ROUND(AVG(HourSpendOnApp))
    FROM (SELECT HourSpendOnApp FROM customer_churn) AS temp
)
WHERE HourSpendOnApp IS NULL;
```

HourSpendOnApp	HourSpendOnApp
NULL	3
3	3
1	1
NULL	3
1	1

- Impute Mean for OrderAmountHikeFromlastYear: The null values in the OrderAmountHikeFromlastYear column were replaced with the mean value of 16.

```
-- Mean for OrderAmountHikeFromlastYear --
UPDATE customer_churn
SET OrderAmountHikeFromlastYear = (
    SELECT ROUND(AVG(OrderAmountHikeFromlastYear))
    FROM (SELECT OrderAmountHikeFromlastYear FROM customer_churn) AS temp
)
WHERE OrderAmountHikeFromlastYear IS NULL;
```

OrderAmountHikeFromlastYear	OrderAmountHikeFromlastYear
17	17
18	18
NULL	16
16	16
13	13

- Impute Mean for DaySinceLastOrder: The null values in the DaySinceLastOrder column were replaced with the mean value of 5.

```
-- Mean for DaySinceLastOrder --
UPDATE customer_churn
SET DaySinceLastOrder = (
    SELECT ROUND(AVG(DaySinceLastOrder))
    FROM (SELECT DaySinceLastOrder FROM customer_churn) AS temp
)
WHERE DaySinceLastOrder IS NULL;
```

DaySinceLastOrder	DaySinceLastOrder
2	2
2	2
0	0
NULL	5
5	5

Mode Imputation:

- Impute Mode for Tenure: The null values in the Tenure column were replaced with the mode value of 1.

```
-- Mode for Tenure --
UPDATE customer_churn
SET Tenure = (
    SELECT Tenure
    FROM (
        SELECT Tenure, COUNT(*) AS cnt
        FROM customer_churn
        WHERE Tenure IS NOT NULL
        GROUP BY Tenure
        ORDER BY cnt DESC
        LIMIT 1
    ) AS temp
)
WHERE Tenure IS NULL;
```

Tenure	Tenure
4	4
NULL	1
NULL	1
0	0
0	0

- Impute Mode for CouponUsed: The null values in the CouponUsed column were replaced with the mode value of 1.

```
-- Mode for CouponUsed --
UPDATE customer_churn
SET CouponUsed = (
  SELECT CouponUsed
  FROM (
    SELECT CouponUsed, COUNT(*) AS cnt
    FROM customer_churn
    WHERE CouponUsed IS NOT NULL
    GROUP BY CouponUsed
    ORDER BY cnt DESC
    LIMIT 1
  ) AS temp
)
WHERE CouponUsed IS NULL;
```

CouponUsed	CouponUsed
0	0
0	0
1	1
0	0
NULL	<u>1</u>

- Impute Mode for OrderCount: The null values in the OrderCount column were replaced with the mode value of 2.

```
-- Mode for OrderCount --
UPDATE customer_churn
SET OrderCount = (
  SELECT OrderCount
  FROM (
    SELECT OrderCount, COUNT(*) AS cnt
    FROM customer_churn
    WHERE OrderCount IS NOT NULL
    GROUP BY OrderCount
    ORDER BY cnt DESC
    LIMIT 1
  ) AS temp
)
WHERE OrderCount IS NULL;
```

OrderCount	OrderCount
NULL	2
1	1
1	1
12	12
1	1

Outlier Handling:

- Rows with WarehouseToHome values greater than **100** were deleted from the customer_churn table as outliers.

```
-- Remove WarehouseToHome Outliers --
DELETE FROM customer_churn
WHERE WarehouseToHome > 100;
```

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome
54123	0	11	Mobile Phone	1	17
54124	0	1	Mobile Phone	3	15
54125	0	26	Computer	3	127
54126	0	18	Phone	1	7
54127	0	1	Phone	3	12

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome
54121	0	12	Mobile Phone	1	17
54122	0	6	Mobile Phone	3	7
54123	0	11	Mobile Phone	1	17
54124	0	1	Mobile Phone	3	15
54126	0	18	Phone	1	7
54127	0	1	Phone	3	12

2.Dealing with Inconsistencies:

- Standardize PreferredLoginDevice: The value Phone was replaced with Mobile Phone in the PreferredLoginDevice column to maintain consistency.

```
-- Standardize PreferredLoginDevice --  
UPDATE customer_churn  
SET PreferredLoginDevice = 'Mobile Phone'  
WHERE PreferredLoginDevice = 'Phone';
```

PreferredLoginDevice	PreferredLoginDevice
Phone	Mobile Phone
Phone	Mobile Phone
Computer	Computer
Phone	Mobile Phone
Phone	Mobile Phone

- Standardize PreferredOrderCat: The value Mobile was replaced with Mobile Phone in the PreferredOrderCat column to maintain consistency.

```
-- Standardize PreferredOrderCat --  
UPDATE customer_churn  
SET PreferredOrderCat = 'Mobile Phone'  
WHERE PreferredOrderCat = 'Mobile';
```

PreferredOrderCat	PreferredOrderCat
Laptop & Accessory	Laptop & Accessory
Mobile	Mobile Phone
Mobile	Mobile Phone
Mobile	Mobile Phone
Others	Others

Data Transformation:

1.Column Renaming:

- Rename PreferredOrderCat to PreferredOrderCat

```
-- Rename PreferredOrderCat to PreferredOrderCat--  
ALTER TABLE customer_churn  
RENAME COLUMN PreferredOrderCat TO PreferredOrderCat;
```

PreferredOrderCat	PreferredOrderCat
Laptop & Accessory	Laptop & Accessory
Mobile Phone	Mobile Phone
Mobile Phone	Mobile Phone
Laptop & Accessory	Laptop & Accessory
Mobile Phone	Mobile Phone

- Rename HourSpendOnApp to HoursSpentOnApp

```
-- Rename HourSpendOnApp to HoursSpentOnApp --  
• ALTER TABLE customer_churn  
  RENAME COLUMN HourSpendOnApp TO HoursSpentOnApp;
```

HourSpendOnApp	HoursSpentOnApp
3	3
3	3
2	2
2	2
3	3

2.Creating New Columns:

- Create ComplaintReceived Column: A new ComplaintReceived column was created with "Yes" for customers who complained and "No" for customers who did not complain.

```
-- Create ComplaintReceived Column --  
ALTER TABLE customer_churn  
ADD ComplaintReceived VARCHAR(3);  
  
UPDATE customer_churn  
SET ComplaintReceived =  
CASE  
  WHEN Complain = 1 THEN 'Yes'  
  ELSE 'No'  
END;
```

ComplaintReceived
Yes
Yes
Yes
No
No

- Create ChurnStatus Column: A new ChurnStatus column was created with "Churned" for churned customers and "Active" for active customers.

```
-- Create ChurnStatus Column --
ALTER TABLE customer_churn
ADD ChurnStatus VARCHAR(10);

UPDATE customer_churn
SET ChurnStatus =
CASE
    WHEN Churn = 1 THEN 'Churned'
    ELSE 'Active'
END;
```

ChurnStatus
Active
Churned
Churned
Active
Active

3. Column Dropping:

- Drop Churn and Complain Columns:

```
-- Drop Churn and Complain Columns --
ALTER TABLE customer_churn
DROP COLUMN Churn,
DROP COLUMN Complain;
```

Columns
CustomerID
<u>Churn</u>
Tenure
PreferredLoginDevice
CityTier
WarehouseToHome
PreferredPaymentMode
Gender
HoursSpentOnApp
NumberOfDeviceRegistered
PreferredOrderCat
SatisfactionScore
MaritalStatus
NumberOfAddress
<u>Complain</u>
OrderAmountHikeFromlastY
CouponUsed

Columns
CustomerID
Tenure
PreferredLoginDevice
CityTier
WarehouseToHome
PreferredPaymentMode
Gender
HoursSpentOnApp
NumberOfDeviceRegistered
PreferredOrderCat
SatisfactionScore
MaritalStatus
NumberOfAddress
OrderAmountHikeFromlastYea
CouponUsed
OrderCount
DaySinceLastOrder
CashbackAmount

Data Exploration and Analysis:

- Count Churned and Active Customers: The number of churned and active customers was calculated by grouping customers based on their churn status.

```
5805      -- Count Churned and Active Customers--
5806 •    SELECT ChurnStatus, COUNT(*) AS CustomerCount
5807      FROM customer_churn
5808      GROUP BY ChurnStatus;
5809
```

Result Grid	Filter Rows:	Export:	Wrap
ChurnStatus	CustomerCount		
Churned	948		
Active	4680		

- Average Tenure and Total Cashback of Churned Customers: The average tenure and total cashback received by churned customers were calculated.

```
5811      -- Average Tenure and Total Cashback of Churned Customers --
5812 •    SELECT
5813          AVG(Tenure) AS AverageTenure,
5814          SUM(CashbackAmount) AS TotalCashback
5815      FROM customer_churn
5816      WHERE ChurnStatus = 'Churned';
5817
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
AverageTenure	TotalCashback		
3.1762	152030		

- Percentage of Churned Customers Who Complained: The percentage of churned customers who submitted complaints was calculated.

```

5818      -- Percentage of Churned Customers Who Complained --
5819 •    SELECT
5820      ROUND(
5821          SUM(CASE WHEN ComplaintReceived = 'Yes' THEN 1 ELSE 0 END)
5822          * 100.0 / COUNT(*), 2
5823      ) AS ComplaintPercentage
5824      FROM customer_churn
5825      WHERE ChurnStatus = 'Churned';
5826

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
ComplaintPercentage			
▶ 53.59			

- City Tier with Highest Churned Customers for Laptop & Accessory: The city tier with the highest number of churned customers who preferred Laptop & Accessory was identified.

```

5827      -- City Tier with Highest Churned Customers for Laptop & Accessory --
5828 •    SELECT CityTier, COUNT(*) AS CustomerCount
5829      FROM customer_churn
5830      WHERE ChurnStatus = 'Churned'
5831      AND PreferredOrderCat = 'Laptop & Accessory'
5832      GROUP BY CityTier
5833      ORDER BY CustomerCount DESC
5834      LIMIT 1;
5835

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
CityTier	CustomerCount			
▶ 3	150			

- Most Preferred Payment Mode Among Active Customers: The most commonly preferred payment mode among active customers was identified.

```

5836      -- Most Preferred Payment Mode Among Active Customers --
5837 •    SELECT PreferredPaymentMode, COUNT(*) AS CustomerCount
5838      FROM customer_churn
5839      WHERE ChurnStatus = 'Active'
5840      GROUP BY PreferredPaymentMode
5841      ORDER BY CustomerCount DESC
5842      LIMIT 1;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PreferredPaymentMode	CustomerCount		
Debit Card	1956		

- Total Order Amount Hike for Single Mobile Phone Customers: The total order amount hike from last year was calculated for single customers who preferred Mobile Phone.

```

5844      -- Total Order Amount Hike for Single Mobile Phone Customers
5845 •    SELECT SUM(OrderAmountHikeFromlastYear) AS TotalOrderAmountHike
5846      FROM customer_churn
5847      WHERE MaritalStatus = 'Single'
5848      AND PreferredOrderCat = 'Mobile Phone';
5849

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
TotalOrderAmountHike			
12177			

- Average Devices Registered by UPI Users: The average number of registered devices among customers using UPI was calculated.

```

5850      -- Average Devices Registered by UPI Users --
5851 •    SELECT AVG(NumberOfDeviceRegistered) AS AverageDevices
5852      FROM customer_churn
5853      WHERE PreferredPaymentMode = 'UPI';
5854

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
AverageDevices			
3.7174			

- City Tier with Highest Number of Customers: The city tier having the highest number of customers was identified.

```

5855 -- City Tier with Highest Number of Customers --
5856 • SELECT CityTier, COUNT(*) AS CustomerCount
5857 FROM customer_churn
5858 GROUP BY CityTier
5859 ORDER BY CustomerCount DESC
5860 LIMIT 1;
5861

```

Result Grid		Filter Rows:	Export:	Wrap C
	CityTier	CustomerCount		
▶	1	3666		

- Gender with Highest Coupon Usage: The gender with the highest total coupon usage was identified.

```

5862 -- Gender with Highest Coupon Usage --
5863 • SELECT Gender, SUM(CouponUsed) AS TotalCoupons
5864 FROM customer_churn
5865 GROUP BY Gender
5866 ORDER BY TotalCoupons DESC
5867 LIMIT 1;

```

Result Grid		Filter Rows:	Export:	Wrap
	Gender	TotalCoupons		
▶	Male	5629		

- Customer Count and Maximum App Hours by Order Category: The customer count and maximum hours spent on the app were calculated for each preferred order category.

```

5869 -- Customer Count and Maximum App Hours by Order Category --
5870 • SELECT
5871     PreferredOrderCat,
5872     COUNT(*) AS CustomerCount,
5873     MAX(HoursSpentOnApp) AS MaxAppHours
5874 FROM customer_churn
5875 GROUP BY PreferredOrderCat;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	PreferredOrderCat	CustomerCount	MaxAppHours	
▶	Laptop & Accessory	2050	5	
	Mobile Phone	2078	5	
	Others	264	4	
	Fashion	826	5	
	Grocery	410	4	

- Total Order Count of Credit Card Users with Maximum Satisfaction: The total order count was calculated for credit card users having the maximum satisfaction score.

```

5877  -- Total Order Count of Credit Card Users with Maximum Satisfaction --
5878  •  SELECT SUM(OrderCount) AS TotalOrderCount
5879  FROM customer_churn
5880  WHERE PreferredPaymentMode = 'Credit Card'
5881  AND SatisfactionScore = (
5882      SELECT MAX(SatisfactionScore)
5883      FROM customer_churn
5884  );
5885

```

Result Grid

TotalOrderCount
1122

- Average Satisfaction Score of Customers Who Complained: The average satisfaction score of customers who submitted complaints was calculated.

```

5886  -- Average Satisfaction Score of Customers Who Complained --
5887  •  SELECT AVG(SatisfactionScore) AS AverageSatisfaction
5888  FROM customer_churn
5889  WHERE ComplaintReceived = 'Yes';

```

Result Grid

AverageSatisfaction
2.9988

- Order Categories of Customers Using More Than 5 Coupons: The preferred order categories of customers who used more than five coupons were identified.

```

5891  -- Order Categories of Customers Using More Than 5 Coupons --
5892  •  SELECT PreferredOrderCat, COUNT(*) AS CustomerCount
5893  FROM customer_churn
5894  WHERE CouponUsed > 5
5895  GROUP BY PreferredOrderCat
5896  ORDER BY CustomerCount DESC;

```

Result Grid

PreferredOrderCat	CustomerCount
Laptop & Accessory	99
Fashion	89
Mobile Phone	45
Grocery	42
Others	28

- Top 3 Order Categories by Average Cashback: The top three preferred order categories based on average cashback amount were identified.

```

5899      -- Top 3 Order Categories by Average Cashback --
5900  •   SELECT
5901          PreferredOrderCat,
5902          AVG(CashbackAmount) AS AverageCashback
5903  FROM customer_churn
5904  GROUP BY PreferredOrderCat
5905  ORDER BY AverageCashback DESC
5906  LIMIT 3;
5907

```

Result Grid		Filter Rows:	Export:	Wrap C
	PreferredOrderCat	AverageCashback		
▶	Others	304.4545		
	Grocery	266.2366		
	Fashion	210.4031		

- Payment Modes of Customers with 10 Months Tenure and More Than 500 Orders: The preferred payment modes were checked for customers with 10 months of tenure and more than 500 orders, but no matching customers were found in the dataset.

```

5908      -- Payment Modes of Customers with 10 Months Tenure and More Than 500 Orders --
5909  •   SELECT DISTINCT PreferredPaymentMode
5910  FROM customer_churn
5911  WHERE Tenure = 10
5912  AND OrderCount > 500;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	PreferredPaymentMode			

- Create Warehouse-to-Home Distance Categories: A distance category column was created based on the Warehouse-to-Home distance, classifying customers into Very Close, Close, Moderate, and Far Distance groups. The churn status breakdown was then displayed for each distance category.

```

5914      -- Warehouse-to-Home Distance Categories --
5915 •    ALTER TABLE customer_churn
5916      ADD DistanceCategory VARCHAR(20);
5917
5918 •    UPDATE customer_churn
5919      SET DistanceCategory =
5920      CASE
5921          WHEN WarehouseToHome <= 5 THEN 'Very Close'
5922          WHEN WarehouseToHome <= 10 THEN 'Close'
5923          WHEN WarehouseToHome <= 15 THEN 'Moderate'
5924          ELSE 'Far'
5925      END;

```

DistanceCategory
Close
Close
Far
Moderate
Moderate
Far

Churn Status by Distance Category:

```

5928      -- Churn Status by Distance Category --
5929 •    SELECT
5930          DistanceCategory,
5931          ChurnStatus,
5932          COUNT(*) AS CustomerCount
5933      FROM customer_churn
5934      GROUP BY DistanceCategory, ChurnStatus
5935      ORDER BY DistanceCategory, ChurnStatus;

```

Result Grid			
Filter Rows:			
Export:			
	DistanceCategory	ChurnStatus	CustomerCount
▶	Close	Active	1696
	Close	Churned	265
	Far	Active	1871
	Far	Churned	498
	Moderate	Active	1106
	Moderate	Churned	184
	Very Close	Active	7
	Very Close	Churned	1

- Find Married Tier 1 Customers with Above-Average Orders: The married customers from City Tier 1 whose order count is higher than the overall average order count were identified.

```
-- Married Tier 1 Customers with Above-Average Orders --
SELECT *
FROM customer_churn
WHERE MaritalStatus = 'Married'
AND CityTier = 1
AND OrderCount > (
    SELECT AVG(OrderCount)
    FROM customer_churn
);
```

CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HoursSpentOnApp	NumberOfDeviceRegistered	PreferredOrderCat
50385	6	Computer	1	12	Debit Card	Male	1	3	Laptop & Accessory
51855	6	Computer	1	12	Debit Card	Male	1	3	Laptop & Accessory
50132	15	Mobile Phone	1	16	Credit Card	Female	2	4	Mobile Phone
50133	13	Mobile Phone	1	8	Credit Card	Male	2	3	Others
50180	12	Mobile Phone	1	6	Credit Card	Male	2	2	Fashion
50229	10	Mobile Phone	1	12	Credit Card	Male	2	2	Laptop & Accessory
50600	0	Mobile Phone	1	23	Credit Card	Female	2	4	Laptop & Accessory

SatisfactionScore	MaritalStatus	NumberOfAddress	OrderAmountHikeFromlastYear	CouponUsed	OrderCount	DaySinceLastOrder	CashbackAmount	ComplaintReceived	ChurnStatus
5	Married	1	11	4	7	8	126	Yes	Active
3	Married	1	16	0	3	9	294	No	Active
2	Married	1	17	3	3	7	196	No	Active
5	Married	1	13	3	4	3	128	Yes	Active
5	Married	1	19	7	7	3	148	No	Active
1	Married	1	13	0	3	8	159	No	Active
1	Married	1	18	2	7	7	125	No	Active

- Create customer_returns Table: A new customer_returns table was created to store customer return details such as Return ID, Customer ID, Return Date, and Refund Amount.

```
5932 • CREATE TABLE customer_returns (
5933     ReturnID INT PRIMARY KEY,
5934     CustomerID INT,
5935     ReturnDate DATE,
5936     RefundAmount INT
5937 );
5938
5939 • SELECT * FROM customer_returns;
```

ReturnID	CustomerID	ReturnDate	RefundAmount
NULL	NULL	NULL	NULL

- Insert Customer Return Records: The provided customer return records were inserted into the customer_returns table for further analysis.

```

5958      -- Insert Customer Return Records --
5959 •    INSERT INTO customer_returns
5960      (ReturnID, CustomerID, ReturnDate, RefundAmount)
5961      VALUES
5962      (1001, 50022, '2023-01-01', 2000),
5963      (1002, 50316, '2023-01-23', 2130),
5964      (1003, 51099, '2023-02-14', 2290),
5965      (1004, 52321, '2023-03-08', 2510),
5966      (1005, 52928, '2023-03-20', 3000),
5967      (1006, 53749, '2023-04-17', 1740),
5968      (1007, 54206, '2023-04-21', 3250),
5969      (1008, 54838, '2023-04-30', 1990);
5970

```

Result Grid				
	ReturnID	CustomerID	ReturnDate	RefundAmount
▶	1001	50022	2023-01-01	2000
	1002	50316	2023-01-23	2130
	1003	51099	2023-02-14	2290
	1004	52321	2023-03-08	2510
	1005	52928	2023-03-20	3000
	1006	53749	2023-04-17	1740
	1007	54206	2023-04-21	3250
	1008	54838	2023-04-30	1990

- Display Return Details of Churned Customers with Complaints: Return details were joined with customer details to display the return information of customers who were both churned and had submitted complaints.

```
-- Display Return Details of Churned Customers with Complaints --
SELECT
    r.ReturnID,
    r.CustomerID,
    r.ReturnDate,
    r.RefundAmount,
    c.Tenure,
    c.PreferredLoginDevice,
    c.CityTier,
    c.WarehouseToHome,
    c.PreferredPaymentMode,
    c.Gender,
    c.PreferredOrderCat,
    c.SatisfactionScore,
    c.MaritalStatus,
    c.OrderCount,
    c.CashbackAmount,
    c.ChurnStatus,
    c.ComplaintReceived
FROM customer_returns r
JOIN customer_churn c ON r.CustomerID = c.CustomerID
WHERE c.ChurnStatus = 'Churned' AND c.ComplaintReceived = 'Yes';
```

ReturnID	CustomerID	ReturnDate	RefundAmount	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	PreferredOrderCat
1002	50316	2023-01-23	2130	0	Computer	2	29	UPI	Female	Fashion
1004	52321	2023-03-08	2510	18	Mobile Phone	3	19	E wallet	Male	Grocery
1006	53749	2023-04-17	1740	1	Mobile Phone	3	31	Credit Card	Female	Laptop & Accessory

SatisfactionScore	MaritalStatus	OrderCount	CashbackAmount	ChurnStatus	ComplaintReceived
5	Married	1	200	Churned	Yes
3	Single	2	237	Churned	Yes
4	Single	11	171	Churned	Yes

Summary

The E-Commerce Customer Churn Analysis project analyzed customer data to identify the factors associated with customer churn. The dataset was cleaned by handling missing values, removing outliers, and standardizing inconsistent values. New columns such as ComplaintReceived and ChurnStatus were created to support further analysis. Various customer attributes, including tenure, payment mode, order category, satisfaction score, coupon usage, cashback, and distance from the warehouse, were analyzed to understand churn patterns. Return information was also combined with customer details to identify returns made by churned customers who had complaints. These findings can help e-commerce businesses understand customer behavior and develop suitable strategies to improve customer retention and satisfaction.